

Да. То, что ты описываешь, — это **RAG для идентификации сотрудников (Employee Identity Resolution RAG)**. Но я бы предложил построить его несколько иначе, чем обычный RAG по документам.

Твоя задача — не просто найти похожий текст, а **однозначно определить, о каком сотруднике идет речь, независимо от того, какое имя, должность, почта или телефон используются**.

Для AI-Breadboard это может стать отдельным универсальным компонентом.

1. Пример задачи

Предположим, у тебя есть один сотрудник:

Атрибут	Значение
ID	EMP-0042
Полное имя	Александр Петров
Имя в HR	Александр Сергеевич Петров
Корпоративная почта	<u>a.petrov@company.com</u>
Дополнительная почта	<u>alex.petrov@gmail.com</u>
Никнейм	Alex
Как называют в компании	Саша, Лёша*
Должность	Senior Developer
Другая должность	Технический руководитель
Телефон	+972-50-123-4567

Атрибут

Значение

Отдел

Engineering

Пример показывает, почему варианты имени необходимо проверять, а не считать автоматически достоверными.

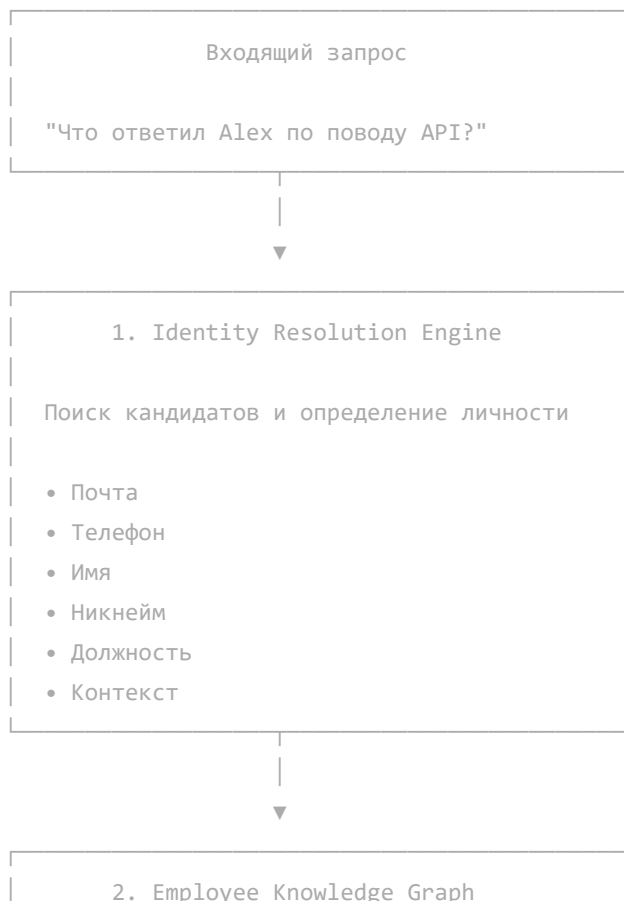
В корпоративной переписке могут встречаться:

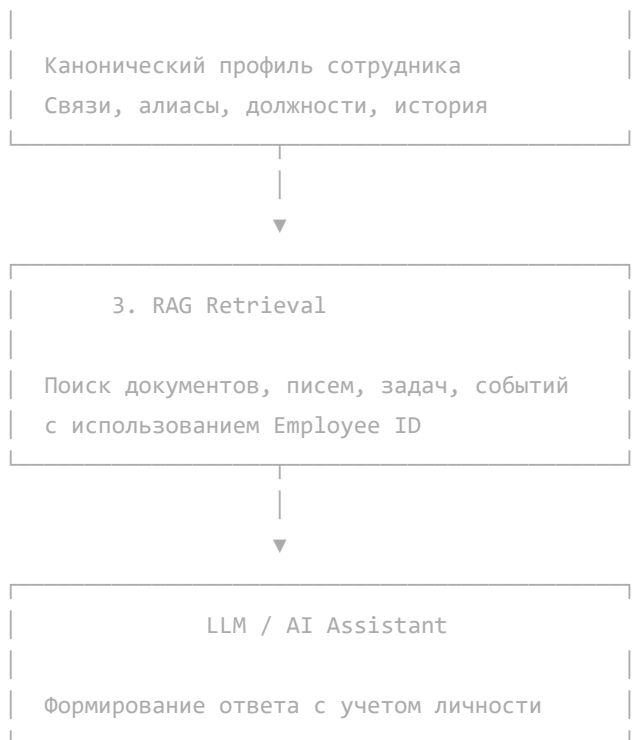
- «Саша посмотрит задачу».
- «Alex, проверь API».
- «Петров согласовал релиз».
- «Отправь отчет на a.petrov».
- «Технический руководитель должен подтвердить».
- «Позвони ему на рабочий номер».

Система должна понимать, что речь может идти об одном человеке, но **не объединять разных сотрудников только из-за совпадения имени или должности.**

2. Главный принцип архитектуры

Я бы разделил систему на три уровня:





Ключевая идея: сначала установить личность, затем искать информацию о ней.

Обычный векторный поиск не гарантирует этого.

3. Канонический профиль сотрудника

Каждому сотруднику присваивается постоянный уникальный идентификатор.

Например:

```
employee_id = EMP-0042
```

Он не должен зависеть от:

- имени;
- должности;
- почтового адреса;
- никнейма;
- подразделения.

Пример структуры

```
{  
  "employee_id": "EMP-0042",  
  "status": "active",  
}
```

```
"canonical_identity": {
  "full_name": "Александр Сергеевич Петров",
  "preferred_name": "Александр"
},

"aliases": [
  {
    "value": "Саша",
    "type": "spoken_name",
    "language": "ru",
    "confidence": 0.95
  },
  {
    "value": "Alex",
    "type": "nickname",
    "language": "en",
    "confidence": 0.90
  },
  {
    "value": "Петров",
    "type": "surname",
    "confidence": 1.0
  }
],

"emails": [
  {
    "address": "a.petrov@company.com",
    "type": "corporate",
    "verified": true
  },
  {
    "address": "alex.petrov@gmail.com",
    "type": "personal",
    "verified": true
  }
],

"phones": [
  {
    "number": "+972501234567",
    "type": "mobile",
    "verified": true
  }
],

"positions": [
  {
    "title": "Senior Developer",
    "department": "Engineering",
    "valid_from": "2024-01-01",
```

```
        "valid_to": null
    }
]
}
```

Это структурированная база идентичности, а не просто набор текстовых документов.

4. Почему одного векторного RAG недостаточно

Предположим, в компании работают:

Сотрудник	Никнейм	Должность
Александр Петров	Alex	Developer
Алексей Петров	Alex	Project Manager
Александр Иванов	Sasha	Developer

Поступает запрос:

«Что Alex сказал на вчерашней встрече?»

Векторный поиск может найти документы всех трех сотрудников.

Даже LLM может ошибиться, если контекст неоднозначен.

Поэтому система должна использовать несколько методов идентификации.

Система должна возвращать

```
{
  "query": "Что Alex сказал на вчерашней встрече?",

  "candidates": [
    {
      "employee_id": "EMP-0042",
      "name": "Александр Петров",
      "score": 0.87
    },
    {
      "employee_id": "EMP-0071",
      "name": "Алексей Петров",
      "score": 0.82
    }
  ]
}
```

```
    }  
  ],  
  
  "resolution": {  
    "status": "ambiguous",  
    "selected_employee_id": null,  
    "reason": "Несколько сотрудников используют никнейм Alex"  
  }  
}  
~  
-CON-
```

Важное правило: при недостатке данных система должна уметь сказать «не удалось однозначно определить сотрудника».

Нельзя заставлять LLM выбирать случайного кандидата.

5. Механизм идентификации

Я бы реализовал каскад поиска.

Уровень 1. Точное совпадение

Самые надежные идентификаторы:

Атрибут	Метод
Employee ID	Точное совпадение
Корпоративная почта	Нормализованное точное совпадение
Проверенный телефон	Нормализованное точное совпадение
Уникальный внешний ID	Точное совпадение

Например:

a.petrov@company.com

→

EMP-0042

Если атрибут достоверно привязан к одному сотруднику, это сильное доказательство.

Уровень 2. Алиасы

Поиск по:

- имени;
- фамилии;
- сокращенному имени;
- никнейму;
- вариантам написания на разных языках;
- корпоративным обращениям.

Например:

"Саша"
"Александр"
"Alex"
"Петров"

Но алиас не обязательно уникален.

Уровень 3. Контекст

Используются:

- отдел;
- должность;
- проект;
- автор письма;
- получатель;
- участники встречи;
- дата события;
- упоминания коллег;
- корпоративная иерархия.

Например:

«Alex из Engineering отправил отчет».

Система использует Engineering для фильтрации кандидатов.

Уровень 4. LLM-дизамбигуация

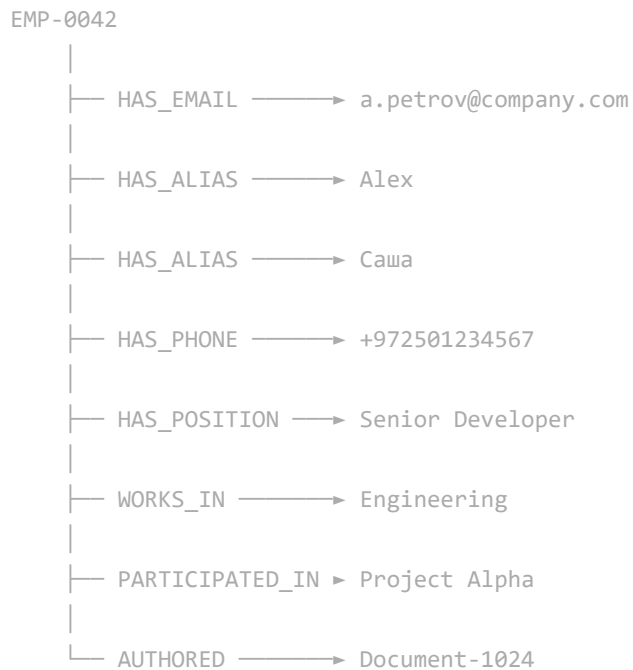
LLM может анализировать найденные варианты и объяснять, какие факты подтверждают или опровергают связь.

Но LLM не должна самостоятельно создавать подтвержденную связь между двумя сотрудниками без достаточных оснований.

6. Employee Knowledge Graph

Здесь я бы использовал не только RAG, но и элементы графа знаний.

Пример:



Граф может хранить связи между:

- сотрудниками;
- почтовыми адресами;
- телефонными номерами;
- документами;
- проектами;
- задачами;
- встречами;
- должностями;
- подразделениями.

Что это дает

Когда найден документ:

Document-1024
Автор: Alex

система может разрешить:

Alex → EMP-0042

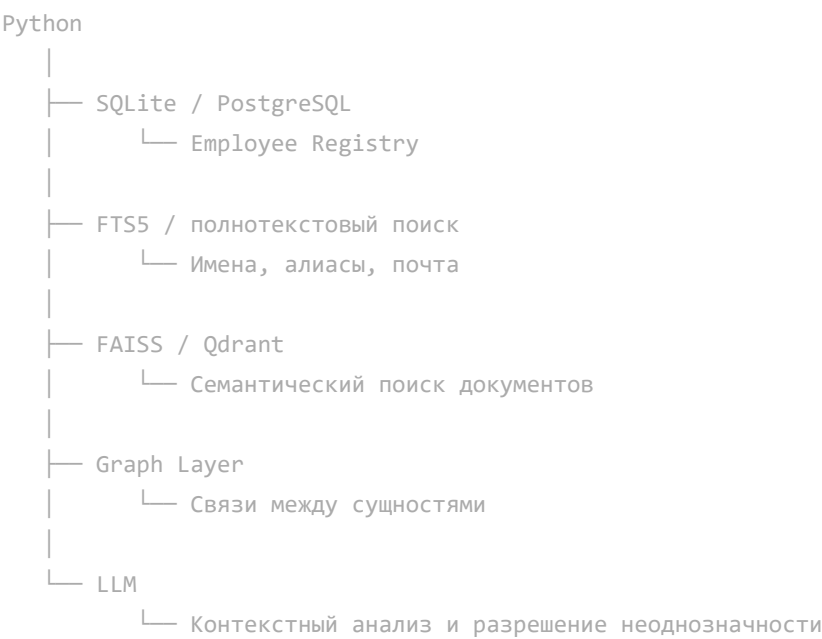
И затем связать документ с каноническим профилем.

7. Гибридное хранение

Для твоего AI-Breadboard я бы предложил следующую структуру.

Компонент	Назначение
Employee Registry	Основная таблица сотрудников
Identity Index	Поиск по именам, почте, телефонам
Alias Registry	Варианты имен и никнеймы
Relationship Graph	Связи между объектами
Vector Index	Семантический поиск документов
Evidence Store	Источники, подтверждающие идентификацию
Resolution Engine	Сопоставление упоминаний с сотрудниками

Возможный стек



На первом этапе необязательно внедрять отдельную графовую базу данных. Можно реализовать связи в SQLite, а при росте проекта добавить специализированное хранилище.

8. Важная функция: история изменений

У сотрудников могут меняться:

- должности;
- отделы;
- фамилии;
- почтовые адреса;
- номера телефонов;
- никнеймы.

Поэтому не стоит просто перезаписывать профиль.

Нужно хранить историю:

```
{
  "employee_id": "EMP-0042",

  "position_history": [
    {
      "title": "Developer",
      "valid_from": "2022-01-01",
      "valid_to": "2024-01-01"
    },
    {
      "title": "Senior Developer",
      "valid_from": "2024-01-01",
      "valid_to": null
    }
  ]
}
```

Это особенно важно для исторических писем и документов.

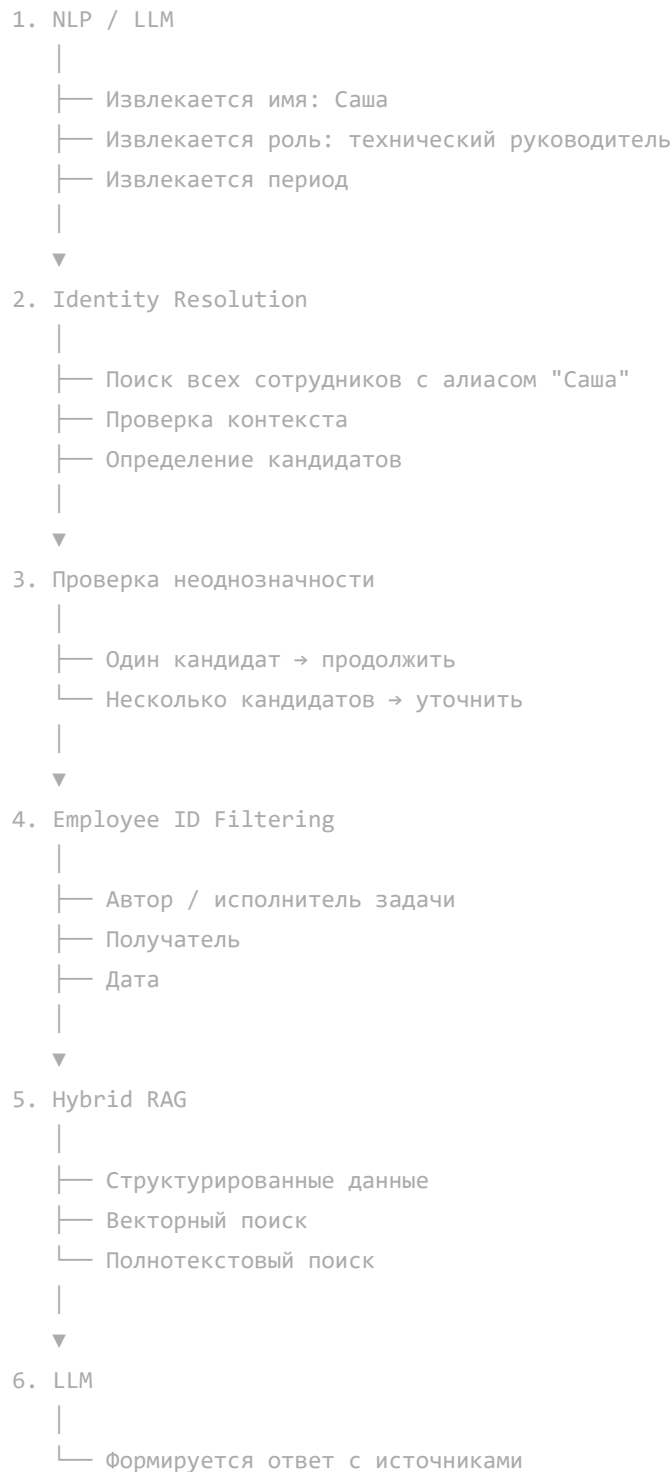
Если документ создан в 2023 году, система должна учитывать должность сотрудника на тот момент, а не только его текущую должность.

9. Как будет работать запрос пользователя

Запрос

«Найди все задачи, которые Саша передал техническому руководителю в прошлом месяце».

Pipeline



10. Отдельно: не путать идентификацию и авторизацию

Это критически важно, если ты собираешься использовать систему в компании.

Определить, что два имени относятся к одному сотруднику, — не значит разрешить пользователю просматривать все данные этого сотрудника.

Я бы разделил:

Identity Resolution

└─ Кто это?

Access Control

└─ Имеет ли пользователь право видеть эти данные?

Например, сотрудник может иметь доступ к рабочим задачам, но не к личному телефону или кадровым документам.

RAG должен учитывать разрешения до формирования контекста для LLM.

11. Что я предлагаю реализовать первым

Для MVP я бы не начинал с полноценного Knowledge Graph.

Фаза 1. Employee Identity Registry

SQLite + Python:

```
employees
employee_emails
employee_phones
employee_aliases
employee_positions
identity_evidence
```

Возможности:

- добавление сотрудников;
- импорт из CSV/Excel/JSON;
- нормализация данных;
- поиск по имени, почте, телефону;
- управление алиасами;
- уникальный Employee ID.

Фаза 2. Identity Resolution Engine

```
resolve_employee(  
    query='Alex',  
    context={  
        'department': 'Engineering',  
        'project': 'Project Alpha'  
    }  
)
```

Python

Run

Возвращает:

```
{
  'status': 'resolved',
  'employee_id': 'EMP-0042',
  'confidence': 0.94,
  'evidence': [...]
```

Python

Run

Или:

```
{
  'status': 'ambiguous',
  'candidates': [...],
  'confidence': None
```

Python

Run

Фаза 3. RAG Integration

Каждому документу присваиваются метаданные:

```
{
  "document_id": "DOC-1001",
  "employee_ids": ["EMP-0042"],
  "source": "email",
  "created_at": "2026-09-20T10:30:00Z"
```

JSON

Затем RAG может искать информацию по конкретному сотруднику, а не только по похожему тексту.

12. Вопросы, от которых зависит архитектура

Чтобы правильно спроектировать систему, мне нужно понять несколько вещей.

1. Откуда будут поступать данные о сотрудниках?

- Excel / CSV?
- Microsoft Active Directory / Entra ID?
- Outlook / Exchange?

- Gmail / Google Workspace?
- CRM / ERP?
- Ручное добавление?
- Переписка и документы?

2. Что ты хочешь идентифицировать?

Только сотрудников компании или также:

- клиентов;
- поставщиков;
- внешних партнеров;
- бывших сотрудников?

3. Как система должна действовать при неоднозначности?

Например, два сотрудника имеют имя Alex.

Варианты:

- Запрашивать уточнение у пользователя.
- Показывать несколько кандидатов.
- Использовать контекст для выбора, но сообщать о неопределенности.
- Запрещать ответ, пока личность не подтверждена.

4. Какой тип данных будет основным?

- Электронная почта.
- Корпоративный чат.
- Документы.
- Телефонные звонки / транскрипции.
- Задачи и проекты.
- Все перечисленное.

5. Как ты хочешь использовать это в AI-Breadboard?

Я вижу два возможных варианта:

Вариант A:

Employee Identity RAG

└─ Отдельный компонент для идентификации

Вариант B:

Enterprise Knowledge RAG

└─ Employee Identity

└─ Documents

└─ Emails

Для твоей задачи я бы начал с **отдельного Employee Identity Resolution Engine**, который затем подключается к любому RAG-провайдеру через стандартный интерфейс AI-Breadboard.

Это позволит использовать один и тот же механизм с Gemini, Ollama, Microsoft Foundry и другими моделями, не связывая идентификацию с конкретной LLM.

    ...  Sources



 Check Point Software Technologies Inc. Ad ...

Secure AI Agents and LLMs

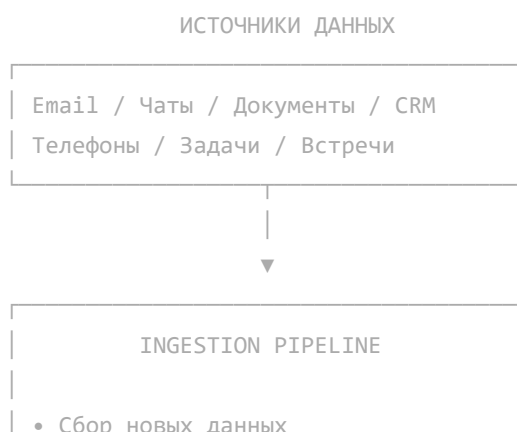
Learn how AI Zero Trust helps secure agents, models, and data.

Ок, но по мере накопления информации мне ее надо собирать в RAG

Да. Тогда тебе нужен **накопительный RAG (Continuous / Incremental RAG)**, который постепенно собирает информацию о сотрудниках, связывает её с конкретными людьми и со временем формирует корпоративную базу знаний.

Причём я бы предложил не просто добавлять документы в векторную базу, а построить систему, которая **накапливает, структурирует, проверяет и уточняет знания о каждом сотруднике**.

1. Общая архитектура





2. Как происходит накопление информации

Допустим, в первый день система получает письмо:

От: a.petrov@company.com

Александр, проверь API до пятницы.

Шаг 1. Извлечение информации

```
{
  "source_type": "email",
  "source_id": "EMAIL-1001",
  "entities": [
    {
      "type": "person",
      "value": "a.petrov@company.com"
    }
  ],
  "facts": [
    {
      "subject": "EMP-0042",
      "predicate": "assigned_task",
      "object": "Проверка API"
    }
  ]
}
```

Система определяет сотрудника по почте и связывает событие с его `employee_id`.

Шаг 2. Добавление в хранилище

```
EMP-0042
├──
├── ASSIGNED_TASK
│   ├──
│   ├── task: Проверка API
│   ├── source: EMAIL-1001
│   ├── date: 2026-09-21
│   └── confidence: verified
```

Шаг 3. Создание RAG-документа

Для семантического поиска можно создать документ:

Employee: EMP-0042

Name: Александр Петров

On 2026-09-21:
A task related to API verification was assigned.
Source: EMAIL-1001

Этот текст индексируется в Vector Store.

Но оригинальное письмо должно сохраняться отдельно. RAG-документ является производным представлением, а не заменой источника.

3. Накапливать нужно не только документы, но и знания

Здесь есть существенное различие.

Обычный RAG

Документ 1 → Embedding → Vector DB
Документ 2 → Embedding → Vector DB
Документ 3 → Embedding → Vector DB

В итоге ты имеешь коллекцию документов.

Твой вариант



Система должна объединять сведения о сотруднике из разных источников.

Например:

Источник	Извлеченный факт
HR	Senior Developer
Email	Участвует в API-проекте
Чат	Помогает коллегам с Python
Задачи	Ответственный за интеграцию
Встреча	Представлял команду
Документ	Автор архитектурного решения

Эти факты не обязательно нужно превращать в окончательные утверждения о компетенциях или должностных полномочиях. Их следует хранить с источниками, датами и статусом подтверждения.

4. Предлагаю разделить RAG на три слоя

Для AI-Breadboard это будет более гибкая архитектура.

Слой A. Raw Knowledge

Сохраняются исходные данные.

```
raw_knowledge/  
├─ emails/  
├─ documents/  
├─ chats/  
├─ meetings/  
└─ tasks/
```

Примеры:

- оригинальные письма;
- документы;
- сообщения;
- транскрипции;
- системные события.

Здесь ничего не теряется.

Слой B. Structured Knowledge

Из исходных данных извлекаются структурированные сведения.

```
employees
employee_aliases
employee_emails
employee_phones
employee_positions
```

```
facts
relationships
projects
tasks
events
```

Пример факта:

```
{
  "fact_id": "FACT-1001",
  "subject_id": "EMP-0042",
  "predicate": "works_on",
  "object": "Project Alpha",

  "valid_from": "2026-01-01",
  "valid_to": null,

  "source_ids": [
    "EMAIL-1001",
    "MEETING-202"
  ],

  "status": "proposed"
}
```

Здесь я бы добавил статус факта:

- `proposed` — извлечен автоматически;
- `confirmed` — подтвержден надежным источником или человеком;
- `rejected` — признан ошибочным;
- `superseded` — заменен более новым фактом.

Это позволяет не выдавать предположение LLM за установленную истину.

Слой C. Semantic RAG

Здесь хранятся embeddings для семантического поиска.

```
Vector Store
├──
├── Employee-related documents
├── Facts
├── Conversations
├── Project context
└── Historical events
```

У каждого фрагмента должны быть метаданные:

```
{
  "chunk_id": "CHUNK-5001",
  "employee_ids": [
    "EMP-0042"
  ],

  "source_id": "EMAIL-1001",
  "knowledge_type": "task",
  "created_at": "2026-09-21T10:30:00Z"
}
```

JSON

Это позволяет применять фильтры по сотрудникам, источникам и датам.

5. Важный момент: RAG должен обновляться

Представим, что сегодня сотрудник Developer, а через полгода становится Team Lead.

Новая информация поступает:

Александр Петров назначен Team Lead.

Система должна:

1. Определить сотрудника.
2. Извлечь изменение должности.
3. Сохранить источник.
4. Добавить новую запись в историю.
5. Не удалять прежнюю должность.
6. Обновить поисковые представления.
7. При необходимости пересчитать связанные сведения.



При запросе:

«Кем был Александр в марте?»

RAG должен учитывать временной контекст.

При запросе:

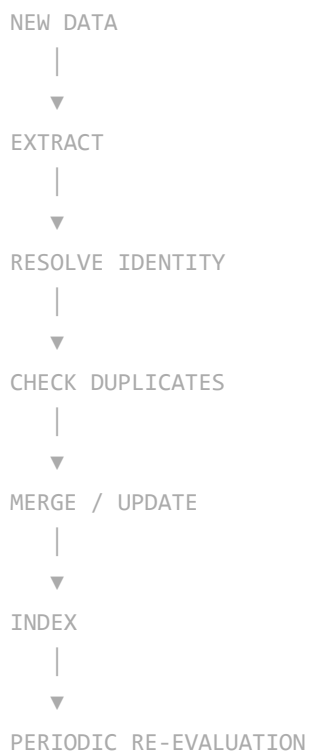
«Кто сейчас Team Lead?»

Нужна актуальная версия данных.

6. Как избежать бесконтрольного роста базы

Накопительный RAG может быстро разрастаться.

Поэтому я бы предусмотрел **циклическую обработку знаний**.



Пример

Сегодня получено:

Alex отвечает за API.

Завтра:

Александр Петров занимается API.

Послезавтра:

Alex из Engineering ведет API-интеграцию.

Система должна попытаться понять, относятся ли сведения к одному сотруднику, но сохранять отдельные источники и не объединять факты без достаточных оснований.

Возможные оптимизации:

- дедупликация документов;
- повторное использование embeddings;
- обновление только изменившихся фрагментов;
- архивация устаревших представлений;
- периодическая проверка противоречий.

7. Как LLM будет использовать накопленные знания

Допустим, ты спрашиваешь:

«Какие задачи обычно выполняет Саша из Engineering?»

Система:

1. Определяет возможных сотрудников с алиасом "Саша".
|
▼
2. Уточняет личность по контексту.
|
▼
3. Получает employee_id.
|
▼
4. Ищет структурированные факты.
|
▼
5. Выполняет семантический поиск документов.
|
▼
6. Проверяет источники и актуальность.



7. Формирует контекст для LLM.

LLM получает:

EMPLOYEE:

Александр Петров

CONFIRMED INFORMATION:

- Senior Developer
- Engineering

PROPOSED INFORMATION:

- Participates in API integration
- Reviews Python code

SOURCES:

- Email-1001
- Task-202
- Meeting-55

И формирует ответ с учетом статуса и источников.

8. Я бы добавил механизм обучения на подтверждениях

Это интересная часть твоей системы.

Предположим, RAG считает:

"Alex" → EMP-0042

Пользователь подтверждает:

Да, Alex — это Александр Петров.

Система может сохранить подтверждение:

```
{
  "alias": "Alex",
  "employee_id": "EMP-0042",
  "verification": {
    "type": "human_confirmation",
    "verified_at": "2026-09-21T12:00:00Z"
  }
}
```

```
}
```

JSON

Но я бы не делал так, чтобы одно подтверждение автоматически связывало этот никнейм со всеми будущими упоминаниями без учета контекста. Никнеймы могут быть общими, а разные источники могут содержать ошибки.

9. Практическая архитектура для AI-Breadboard

Я бы выделил отдельный сервис:

```
ai-breadboard/  
|  
├─ services/  
|   ├── identity/  
|   |   ├── resolver.py  
|   |   ├── aliases.py  
|   |   └── profiles.py  
|   |  
|   ├── knowledge/  
|   |   ├── ingestion.py  
|   |   ├── extraction.py  
|   |   ├── facts.py  
|   |   └── consolidation.py  
|   |  
|   └── rag/  
|       ├── indexing.py  
|       ├── retrieval.py  
|       └── query.py  
|  
├─ storage/  
|   ├── employees.db  
|   ├── knowledge.db  
|   └── vector_store/  
|  
└─ workers/  
    ├── ingest_worker.py  
    ├── embedding_worker.py  
    └── consolidation_worker.py
```

Это пример логического разделения, а не обязательная структура каталогов.

Основные API

```
POST /api/v1/knowledge/ingest
```

Добавление новых данных.

POST /api/v1/knowledge/extract

Извлечение фактов.

POST /api/v1/identity/resolve

Определение сотрудника.

POST /api/v1/knowledge/consolidate

Объединение и проверка знаний.

POST /api/v1/rag/query

Поиск и формирование контекста.

Мой вывод

Тебе нужен не просто Employee RAG, а **накопительная система корпоративных знаний с разрешением идентичности**.

Я бы строил её по принципу:

Raw Data → Identity Resolution → Structured Facts → Evidence → Vector Index → Hybrid Retrieval → LLM

При этом важно, чтобы система сохраняла историю, источники, неопределенность и возможность исправления ошибок.

Следующий важный вопрос

Откуда именно будет поступать информация о сотрудниках?

Я бы хотел уточнить это, потому что архитектура накопления сильно зависит от источников:

1. **Email** — Outlook / Exchange / Gmail?
2. **Корпоративные чаты** — Teams / Slack / WhatsApp?
3. **Документы** — PDF, Word, Excel, внутренние базы?
4. **Телефонные разговоры** — записи и транскрипции?
5. **Ручные данные** — пользователь сам рассказывает AI о сотрудниках?
6. **Все источники одновременно?**

Если ты планируешь использовать **все источники**, то следующим шагом стоит спроектировать полноценный **Knowledge Ingestion Pipeline** с очередью событий, дедупликацией и инкрементальным обновлением RAG.



HookMyApp

Ad



Let your AI agent set up WhatsApp

Paste one prompt into Claude Code, Codex or Cursor. WhatsApp and Instagram...

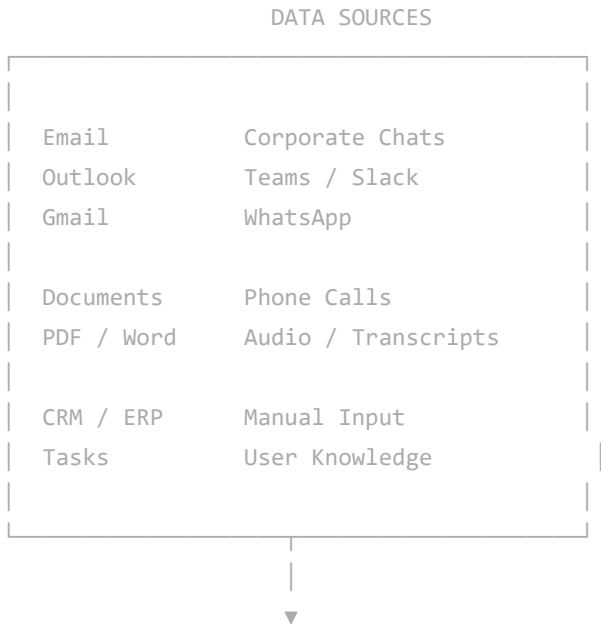
Все источники одновременно

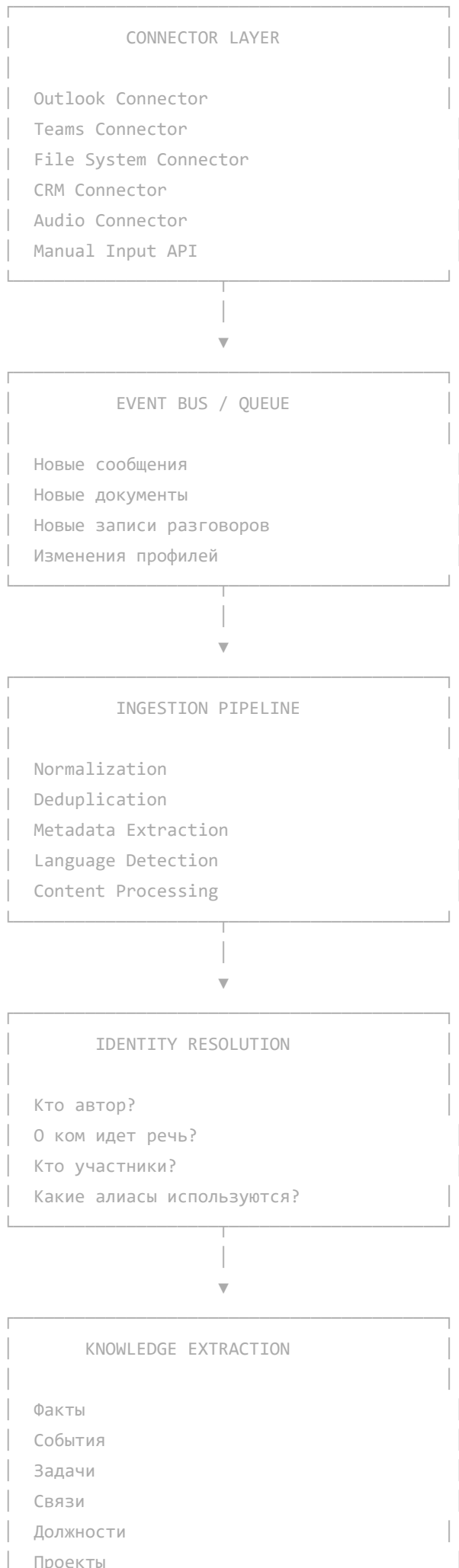
Тогда я бы проектировал для тебя **универсальную корпоративную систему накопления знаний (Enterprise Knowledge Intelligence Platform)**, в которой информация из всех источников постепенно объединяется в единое хранилище, привязанное к конкретным сотрудникам.

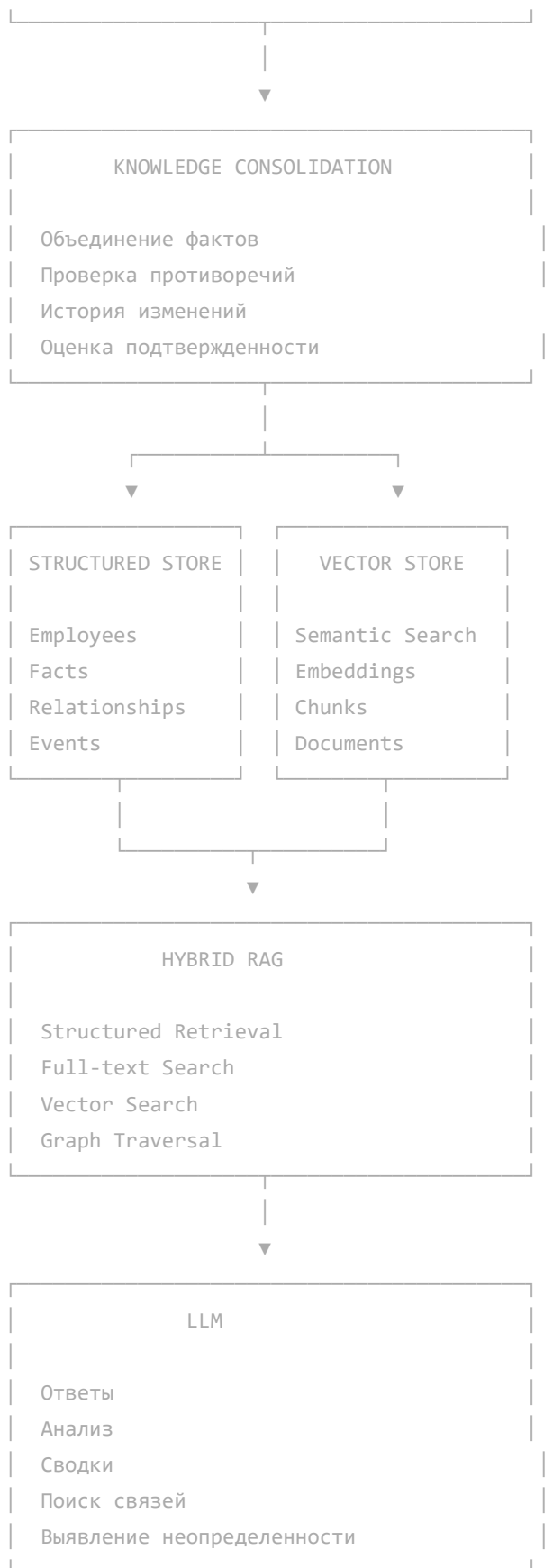
Это уже не просто RAG для поиска документов. Это система, которая строит **динамическую карту сотрудников, их деятельности, связей и корпоративных знаний**.

Рассмотрим архитектуру, которую можно реализовать в AI-Breadboard.

1. Общая архитектура



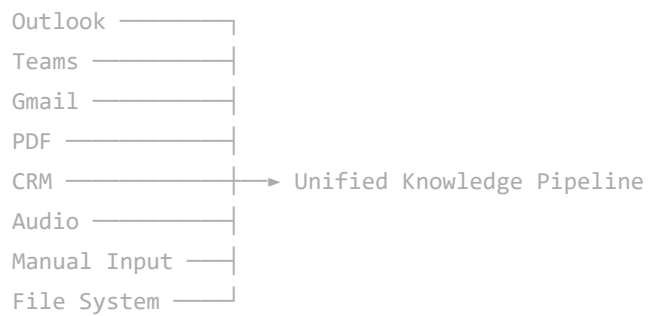




2. Главное решение: единый Ingestion Pipeline

Не стоит создавать отдельный RAG для каждого источника.

Например:



Каждый источник преобразуется в единый внутренний формат.

Унифицированное событие

```
{
  "event_id": "EVT-10001",
  "source": {
    "type": "email",
    "system": "outlook",
    "external_id": "MSG-12345"
  },
  "event_type": "message_received",
  "occurred_at": "2026-09-21T10:30:00Z",

  "participants": [
    {
      "external_identity": "a.petrov@company.com",
      "role": "sender"
    }
  ],

  "content": {
    "text": "Проверь API до пятницы.",
    "language": "ru"
  },

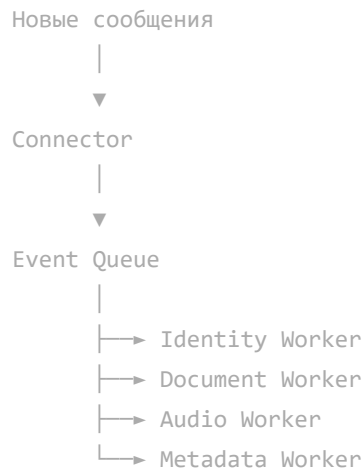
  "metadata": {
    "department": "Engineering",
    "classification": "internal"
  }
}
```

Этот формат не зависит от того, поступила информация из Outlook, Teams или CRM.

3. Очередь событий

Поскольку данные будут поступать постоянно, обработку лучше отделить от приема.

Например



В качестве очереди можно рассмотреть:

Технология	Назначение
SQLite Queue	Простой локальный MVP
Redis Streams	Локальная асинхронная обработка
RabbitMQ	Надежная маршрутизация сообщений
NATS JetStream	Событийная архитектура
Kafka	Большие потоки данных и масштабирование

Для первого локального прототипа я бы начал с SQLite + фоновых Python workers. Не обязательно сразу внедрять Kafka.

Важные свойства:

- повторная обработка после сбоя;
- уникальный `event_id`;
- статус обработки;
- журнал ошибок;
- повторные попытки;
- идемпотентность.

4. Разные источники требуют разных обработчиков

Единый Pipeline не означает, что все данные обрабатываются одинаково.

Email

Извлекаются:

- отправитель;
- получатели;
- копии;
- тема;
- текст;
- вложения;
- время;
- цепочка переписки.

Корпоративные чаты

Извлекаются:

- автор сообщения;
- участники;
- канал;
- ответы;
- реакции;
- ссылки на сообщения.

Документы

Извлекаются:

- автор;
- метаданные;
- текст;
- таблицы;
- заголовки;
- версии документа.

Телефонные разговоры

Pipeline:

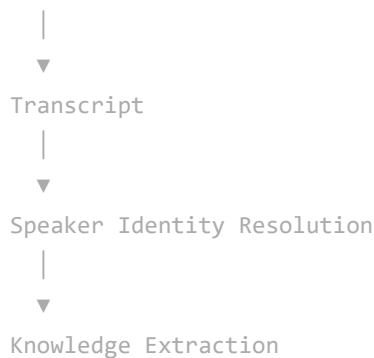
Audio



Speech-to-Text



Speaker Diarization



Здесь есть отдельная сложность: **определение говорящего по голосу не равно однозначной идентификации сотрудника**. Нужны подтвержденные связи с участниками разговора и контроль ошибок.

Ручной ввод

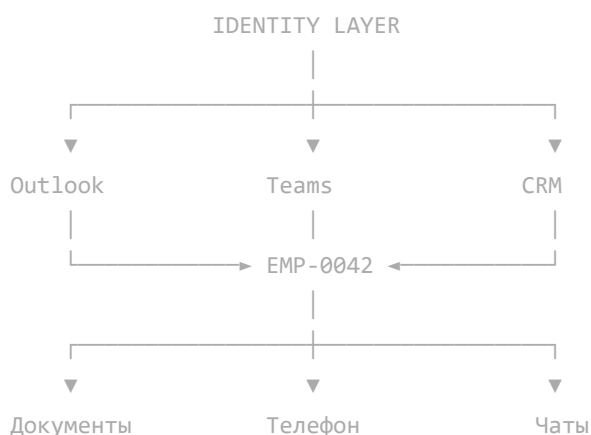
Например, пользователь говорит:

«Саша из бухгалтерии отвечает за отчетность».

Система извлекает предположительный факт, но должна отличать утверждение пользователя от подтвержденных кадровых данных.

5. Единый Employee Identity Layer

Все источники должны обращаться к одному реестру идентичности.



Пример

В Outlook:

a.petrov@company.com

В Teams:

alex.petrov

В CRM:

User ID: 781

В документах:

Александр Петров

Система связывает эти идентификаторы с:

EMP-0042

Но каждую связь следует хранить с источником, способом установления и уровнем доверия.

6. Накопление знаний: Fact Store

Это один из наиболее важных компонентов.

Не нужно хранить всю информацию только как embeddings.

Я бы использовал структуру:

```
Knowledge Store
├── Sources
│   └── Оригинальные данные
├── Facts
│   └── Извлеченные утверждения
├── Entities
│   └── Сотрудники / проекты / задачи
├── Relationships
│   └── Связи между сущностями
├── Events
│   └── Действия и изменения
└── Evidence
    └── Подтверждающие источники
```

Пример накопления

Первое письмо:

Александр занимается интеграцией API.

Второе письмо:

Алех подготовил документацию API.

Третья задача:

Ответственный за API-интеграцию — Александр Петров.

В базе постепенно формируются:



При этом каждое утверждение хранится отдельно, с источником и временным контекстом.

Нельзя считать повторение одного и того же утверждения в нескольких письмах независимым доказательством: письма могут копировать друг друга.

7. Гибридный поиск

Я бы не делал ставку только на Vector RAG.

Тебе нужен **Hybrid Retrieval**.

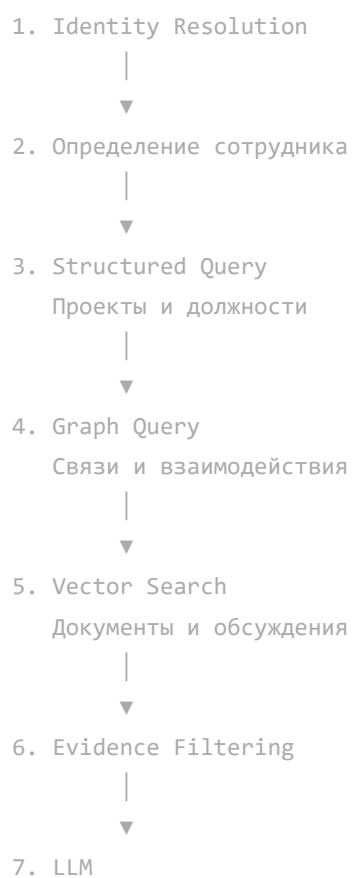
Тип поиска	Для чего
Exact Search	Email, Employee ID, телефон
Full-Text Search	Имена, термины, фразы
Vector Search	Семантически похожие документы
Structured Query	Факты, даты, должности

Тип поиска	Для чего
Graph Search	Связи между сотрудниками
Temporal Search	История изменений

Пример запроса

«Какие проекты ведет Саша из Engineering и с кем он взаимодействует?»

Pipeline:



LLM получает не просто набор похожих фрагментов, а контекст из разных хранилищ.

8. Контур проверки и консолидации

По мере накопления данных будут появляться противоречия.

Например:

Источник А:
Александр Петров — Developer

Источник В:
Александр Петров — Team Lead

Это может быть:

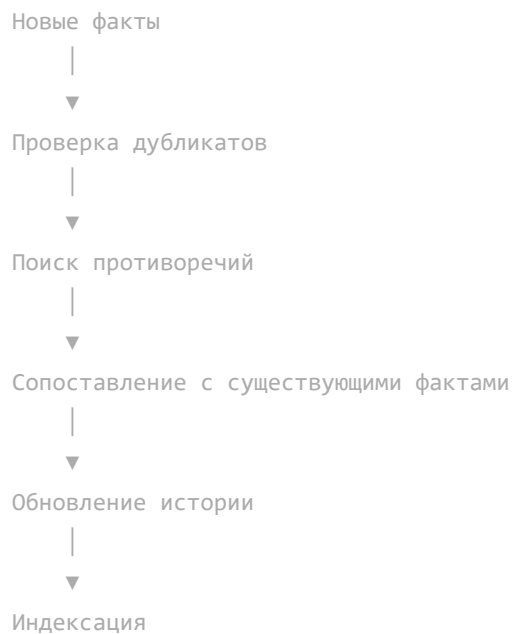
- изменение должности;
- устаревший документ;
- ошибка в источнике;
- разные сотрудники с одинаковым именем.

Система должна сохранять обе записи, а затем анализировать:

Fact A
├─ valid_from
├─ valid_to
├─ source
└─ status

Fact B
├─ valid_from
├─ valid_to
├─ source
└─ status

Knowledge Consolidation Worker



LLM может помогать анализировать противоречия, но окончательное подтверждение важных сведений должно зависеть от политики источников и, при необходимости, человека.

9. Предлагаемая структура компонентов AI-Breadboard

Я бы разделил систему на независимые модули.

```
ai-breadboard/
|
├─ connectors/
|   ├── outlook/
|   ├── teams/
|   ├── gmail/
|   ├── filesystem/
|   ├── crm/
|   └─ audio/
|
├─ ingestion/
|   ├── events/
|   ├── normalization/
|   ├── deduplication/
|   └─ processing/
|
├─ identity/
|   ├── registry/
|   ├── resolution/
|   ├── aliases/
|   └─ verification/
|
├─ knowledge/
|   ├── extraction/
|   ├── facts/
|   ├── relationships/
|   ├── consolidation/
|   └─ temporal/
|
├─ retrieval/
|   ├── structured/
|   ├── fulltext/
|   ├── vector/
|   ├── graph/
|   └─ hybrid/
|
├─ storage/
|   ├── relational/
|   ├── vector/
|   ├── object/
|   └─ evidence/
|
└─ workers/
    ├── ingestion_worker.py
    ├── extraction_worker.py
    ├── resolution_worker.py
    └─ consolidation_worker.py
```

Это логическая организация. Реальные каталоги можно адаптировать под существующую структуру AI-Breadboard.

10. Где использовать LLM

Я бы не передавал LLM абсолютно все операции.

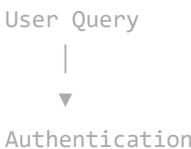
Операция	Предпочтительный подход
Нормализация почты	Код
Нормализация телефонов	Код
Поиск точного ID	База данных
Дедупликация файлов	Хеши
Извлечение фактов	LLM + структурированная схема
Идентификация по контексту	Правила + поиск + LLM
Проверка противоречий	Правила + LLM
Семантический поиск	Embeddings
Финальный ответ	LLM

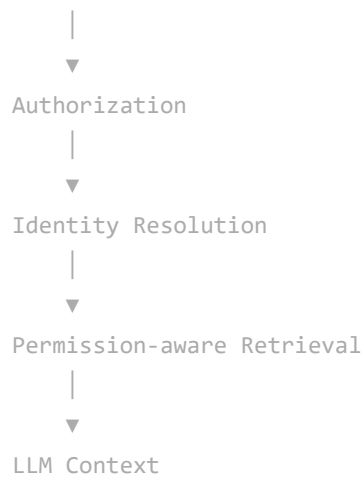
Для локальной работы можно использовать Ollama, а для более сложного извлечения — Gemini или Microsoft Foundry, если это разрешено политикой обработки данных.

11. Безопасность: особенно важно для корпоративных данных

При объединении всех источников появляется риск создать единый индекс, который раскрывает больше информации, чем разрешено конкретному пользователю.

Поэтому я бы заложил **Access Control до Retrieval**.





Необходимо учитывать:

- права доступа к исходному документу;
- конфиденциальность переписки;
- персональные данные;
- ограничения на передачу информации внешним LLM;
- удаление и отзыв доступа;
- аудит запросов и ответов.

Например, наличие сотрудника в базе не означает, что любой пользователь имеет право просматривать его личный телефон, кадровые сведения или приватные сообщения.

12. С чего начать реализацию

Я бы разделил разработку на четыре этапа.

Этап 1. Unified Knowledge Model

Создать:

- Employee ;
- Source ;
- Document ;
- Fact ;
- Relationship ;
- Event ;
- Evidence .

На этом этапе не требуется подключать все источники.

Этап 2. Ingestion Framework

Создать общий интерфейс коннекторов:

```
class BaseConnector:
    asvnc def fetch changes(self) -> list[dict]:

Python

    async def normalize(self, item: dict) -> dict:
        ...
```

Run

Конкретные коннекторы реализуют собственную работу с Outlook, Teams, файлами и другими источниками.

Этап 3. Identity + Knowledge Processing

Добавить:

- сопоставление сотрудников;
- извлечение фактов;
- историю;
- проверку противоречий;
- сохранение доказательств.

Этап 4. Hybrid RAG

Подключить:

- полнотекстовый поиск;
- векторный поиск;
- фильтрацию по Employee ID;
- фильтрацию по разрешениям;
- LLM-контекст.

Моя рекомендация по архитектуре

Для твоего проекта я бы выбрал следующую стратегию:

Не строить один огромный RAG, в который складывается всё подряд.

Строить:

```
Unified Enterprise Knowledge Platform
|
├─ Identity Registry
├─ Source Registry
├─ Raw Data Store
├─ Fact Store
└─ Relationship Store
```

- └ Evidence Store
- └ Vector Index
- └ Hybrid Retrieval API

RAG в этой архитектуре становится поисковым и генеративным слоем над корпоративными знаниями, а не единственным хранилищем.

Следующий шаг

Я предлагаю спроектировать конкретную схему базы данных и событийную модель `ai-knowledge-core`.

В ней мы определим:

1. Таблицы сотрудников и идентификаторов.
2. Унифицированную структуру источников.
3. Схему фактов и отношений.
4. События ingestion.
5. Статусы обработки.
6. Хранение доказательств.
7. API для подключения любого источника.
8. Связь с существующим RAG в AI-Breadboard.

Это позволит сначала создать надежное ядро, а затем подключать Outlook, Teams, Gmail, документы, CRM и аудиозаписи независимо друг от друга.



Для одновременных источников нужен слой обновления и защиты данных через Zero Trust.



 Check Point Software Technologies Inc. Ad ...

Secure AI Agents and LLMs

Learn how AI Zero Trust helps secure agents, models, and data.

ChatGPT can make mistakes. Check important info.